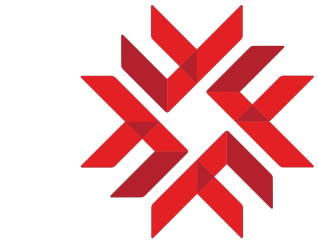
Natural Language Processing 2

Building an LLM from Scratch—Ollama and User Interface

Fatemeh Dehrouyeh



FANSHAWE

Contents

1. Ollama
   - Basic Text Generation
   - Chat-Style Example
   - Keep a Conversation Going
2. User Interface
   - Streamlit
   - Gradio
   - chainlit

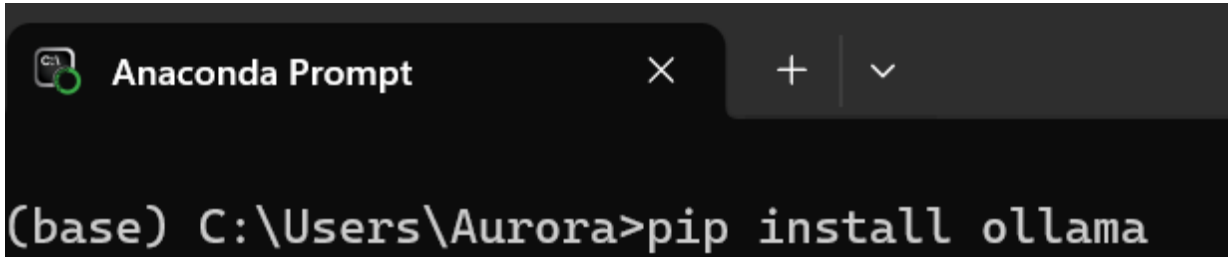
Current Section

1. Ollama
   - Basic Text Generation
   - Chat-Style Example
   - Keep a Conversation Going
2. User Interface

Ollama

Instead of manually writing the urllib.request code (as in the previous chapter), we can install the official Ollama Python library and write much simpler code!

We need to code on our local machine using JupyterLab, for example.
The Ollama app must already be installed on your machine. The first step is to install the Ollama library.
Open Anaconda Prompt and run:



In JupyterLab, you can also run:
!pip install ollama

Using the Llama model through Ollama

```python
import ollama

response = ollama.generate(
    model="llama3.2",
    prompt="Explain byte pair encoding in one simple paragraph."
)

print(response["response"])
```

```
Byte Pair Encoding (BPE) is a technique used for text compression and preprocessing, whe
re it groups together frequently occurring character pairs to reduce the amount of data.
The algorithm works by iteratively identifying common character pairs that appear togeth
er frequently, such as "th" or "ch", and merging them into single characters, effectivel
y reducing the vocabulary size and minimizing redundancy in the text. This process is re
peated until no further pairs can be identified, resulting in a compressed representatio
n of the original text with reduced character frequency.
```

If a model is already downloaded, you can use it directly. If it is not downloaded, you need to pull the model first using your terminal, Command Prompt, PowerShell, or Anaconda Prompt.

Example: ollama pull model_name

For example: ollama pull llama3.2

Current Subsection

1. Ollama
   - Basic Text Generation
   - Chat-Style Example
   - Keep a Conversation Going
2. User Interface

1. Basic Text Generation

Use `generate()` when you want to send one prompt and get one answer.

```python
import ollama

response = ollama.generate(
    model="llama3.2",
    prompt="What meal is best for health?"
)

print(response["response"])
```

```
The best meal for health is one that is nutrient-dense, balanced, and varied. Here are s
ome general guidelines for a healthy meal:

1. **Focus on whole foods**: Choose whole, unprocessed foods like vegetables, fruits, wh
ole grains, lean proteins, and healthy fats.
2. **Include a variety of colors**: Aim for a rainbow of colors on your plate to ensure
you're getting a range of vitamins and minerals.
3. **Incorporate plant-based protein sources**: Include plant-based protein sources like
beans, lentils, tofu, and tempeh to support heart health and weight management.
4. **Select lean proteins**: Choose lean protein sources like poultry, fish, and legumes
instead of red meat or processed meats.
5. **Healthy fats are essential**: Nuts, seeds, avocados, and olive oil are all great so
urces of healthy fats that support heart health.
6. **Limit added sugars**: Aim to limit foods with added sugars, which can increase the
risk of chronic diseases like diabetes and obesity.
7. **Stay hydrated**: Drink plenty of water throughout the day, aiming for at least 8 cu
ps (64 ounces) daily.
```

This sends one prompt and gets one answer.

Current Subsection

1. Ollama
   - Basic Text Generation
   - Chat-Style Example
   - Keep a Conversation Going
2. User Interface

2. Chat-Style Example

Use `chat()` when you want a conversation with roles like `system`, `user`, and `assistant`.

```
response = ollama.chat(
    model="llama3.2",
    messages=[
        {
            "role": "system",
            "content": "You are a helpful teaching assistant. Explain clearly and simply."
        },
        {
            "role": "user",
            "content": "What is Ollama?"
        }
    ]
)

print(response["message"]["content"])
```

```
Ollama can refer to two different things:

1. **Oloma**: In some cultures, Oloma is a type of tree or shrub that is commonly found in tropi
cal regions.

2. **Oloma (in video games)**: However, I think you might be referring to "Ollama" in the contex
t of video games, specifically the popular game "Overcooked".

In Overcooked, Ollama is not directly related to any specific character or item. Instead, it's a
term that players use to describe a type of food or ingredient used in the game.

If you have any more information about what you're looking for, feel free to ask!
```

Ollama's chat API uses a list of messages, where each message has a role and content.

Chat Roles

Each message has a role.

The role tells the model who is speaking and how to use the message.

- **system**: tells the model how to behave
  You are a helpful teaching assistant.
- **user**: asks the question
  What is Ollama?
- **assistant**: stores the model's previous answer
  Ollama lets you run AI models locally.
- **tool**: gives information from an external tool
  The current temperature is 18 degrees Celsius.

Another Example—Assistant Role

```python
import ollama

messages = [
    {
        "role": "system",
        "content": "You are a helpful teaching assistant. Explain clearly and simply."
    },
    {
        "role": "user",
        "content": "What is Python?"
    },
    {
        "role": "assistant",
        "content": "Python is a programming language. People use it for data analysis, websites, automation, and AI."
    },
    {
        "role": "user",
        "content": "Can you give me one simple Python example?"
    }
]

response = ollama.chat(
    model="llama3.2",
    messages=messages
)

print(response["message"]["content"])
```

```
Here's a simple example:

**Print Your Name**

```
name = "John"
print("Hello, my name is " + name)
```

When you run this code, it will print: "Hello, my name is John"

This shows how to store a value (your name) in a variable and then use that value in a sentence.
```

The *assistant* message contains a previous answer from the model, or an example answer that we write ourselves to show the model the conversation history.

Note: The model itself does not automatically maintain a permanent record of every message. In the API, the developer controls how conversation history is stored and supplied. Some API features can also manage conversation state.

Another Example—Assistant Role

```
messages = [
    {
        "role": "system",
        "content": "You are a helpful teaching assistant."
    },
    {
        "role": "user",
        "content": "What is Python?"
    }
]

response = ollama.chat(
    model="llama3.2",
    messages=messages
)

assistant_reply = response["message"]["content"]

messages.append({
    "role": "assistant",
    "content": assistant_reply
})

messages.append({
    "role": "user",
    "content": "Can you give me one example?"
})

response = ollama.chat(
    model="llama3.2",
    messages=messages
)

print(response["message"]["content"])
```

This code shows how to save the model's first answer in the conversation history, then send it back with a follow-up question so the model can respond with context.

Current Subsection

1. Ollama
   - Basic Text Generation
   - Chat-Style Example
   - Keep a Conversation Going
2. User Interface

3. Keep a Conversation Going

This keeps the full conversation history in the `messages` list.

```python
import ollama

messages = [
    {"role": "system", "content": "You are a concise AI tutor."}
]

while True:
    user_input = input("You: ")

    if user_input.lower() in ["exit", "quit"]:
        break

    messages.append({"role": "user", "content": user_input})

    response = ollama.chat(
        model="llama3.2",
        messages=messages
    )

    assistant_reply = response["message"]["content"]
    messages.append({"role": "assistant", "content": assistant_reply})

    print("\nAssistant:", assistant_reply, "\n")
```

The assistant role identifies the model's previous responses and allows the model to recognize them as part of the conversation history.

Note: The conversation cannot exceed the model's context window.

Result

```
You:  What is OpenClaw?

Assistant: OpenCLAW (Claw) is an open-source, cloud-based platform for designing and implementin
g complex cryptographic algorithms and protocols. It's designed to accelerate the development of
secure systems and applications.

OpenCLAW provides tools for algorithm design, implementation, and testing, allowing users to foc
us on the logic of their cryptographic schemes rather than the underlying hardware or software i
nfrastructure.

Some key features of OpenCLAW include:

1. Cryptographic framework: Provides a modular and flexible framework for designing cryptographi
c algorithms.
2. Hardware acceleration: Allows developers to accelerate their algorithms using dedicated hardw
are components.
3. Cloud-based deployment: Enables easy deployment and testing of cryptographic schemes on vario
us cloud platforms.

OpenCLAW aims to simplify the process of developing secure systems and protocols, making it more
accessible to researchers, developers, and organizations across industries.

You:
```

Current Section

1. Ollama
2. User Interface
   - Streamlit
   - Gradio
   - chainlit

Adding a User Interface

Instead of writing the prompt inside Python code, we can create a small web app.

Good options:

- **Streamlit**: easiest for beginners. Best when you want a simple web app with buttons, text boxes, charts, tables, and basic AI interaction.
- **Chainlit**: best for chatbot-style AI apps. Use it when you want a ChatGPT-like interface with message history and multi-turn conversations.
- **Gradio**: good for quick demos. Best when you want to test a model with simple inputs and outputs, such as text, image, audio, or file upload.
- **LitServe**: better for serving models as an API. Use it when you want to deploy a model so other apps can send requests to it.

Current Subsection

1. Ollama
2. User Interface
   - Streamlit
   - Gradio
   - chainlit

Streamlit App with Ollama

Save this code as `app.py`.

```python
import streamlit as st
import ollama

st.title("Ollama Chat App")

user_prompt = st.text_area(
    "Ask a question:",
    "Explain byte pair encoding in one simple paragraph."
)

if st.button("Generate"):
    response = ollama.generate(
        model="llama3.2",
        prompt=user_prompt
    )

    st.write(response["response"])
```

Run it in the terminal:

```
streamlit run app.py
```

Installing Streamlit

First, install Streamlit in your terminal:

```
python -m pip install streamlit ollama
```

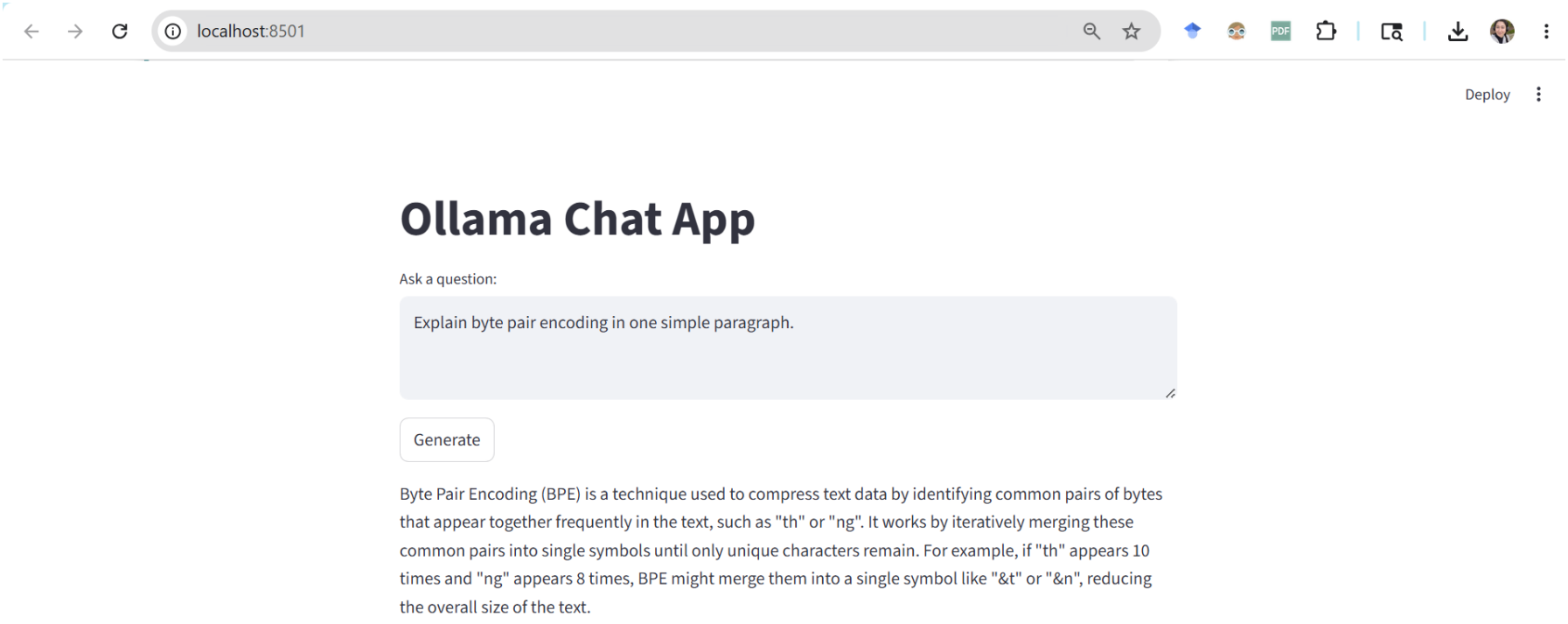
Then run the app:

```
python -m streamlit run C:\Users\Aurora\Desktop\NLP2\app.py
```

Make sure Ollama is already running and the model is downloaded:

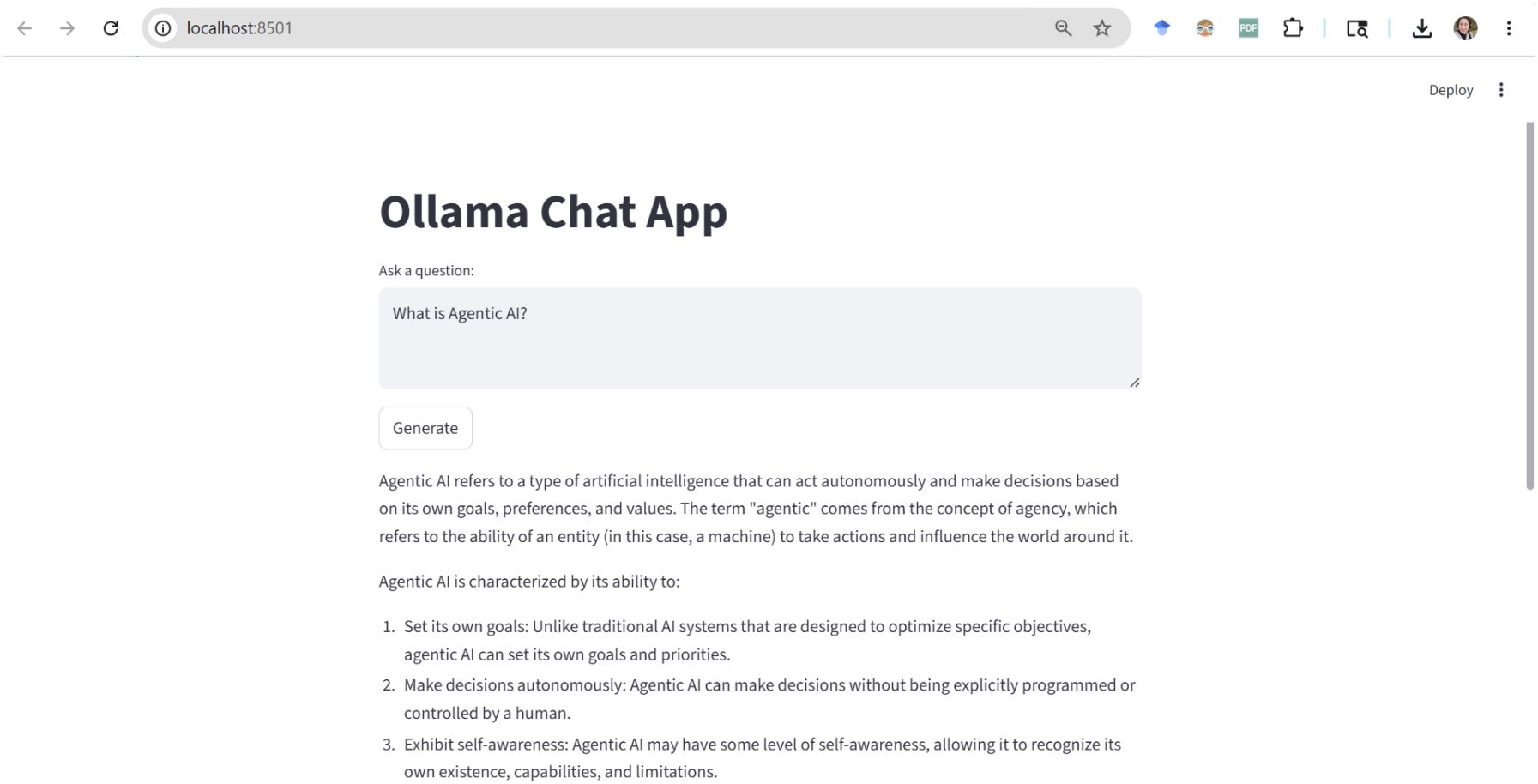
```
ollama pull llama3.2
```

The App Opens in the Browser



Changing the Prompt

You can input any prompt now:



You could Also Install Streamlit Using Anaconda Prompt

For Installing Streamlit in Anaconda Prompt:

```
pip install streamlit
```

Then run the app:

```
streamlit run app.py
```

Replace `app.py` with the full file path.

Current Subsection

1. Ollama
2. User Interface
   - Streamlit
   - Gradio
   - chainlit

What is Gradio?

Gradio is a Python library for building simple web interfaces.

It is useful when we want to test a model quickly.

Good for:

- Quick AI demos
- Text input and output
- Image, audio, and file inputs
- Sharing a model with others

Gradio is simple, fast, and beginner-friendly.

Installing Gradio

First, open a terminal and install Gradio:

```
python -m pip install gradio
```

Make sure Ollama is already running.

Also make sure the model is downloaded:

```
ollama pull llama3.2
```

Gradio App with Ollama

Save this code as `app1.py`.

```python
import gradio as gr
import ollama

def chat_with_model(prompt):
    response = ollama.generate(
        model="llama3.2",
        prompt=prompt
    )
    return response["response"]

demo = gr.Interface(
    fn=chat_with_model,
    inputs="text",
    outputs="text",
    title="Ollama Gradio App",
    flagging_dir=r"C:\Users\Aurora\Desktop\NLP2\flagged_outputs"
)

demo.launch()
```

Running the Gradio App

Run the app from the terminal:

```
python C:\Users\Aurora\Desktop\NLP2\app1.py
```

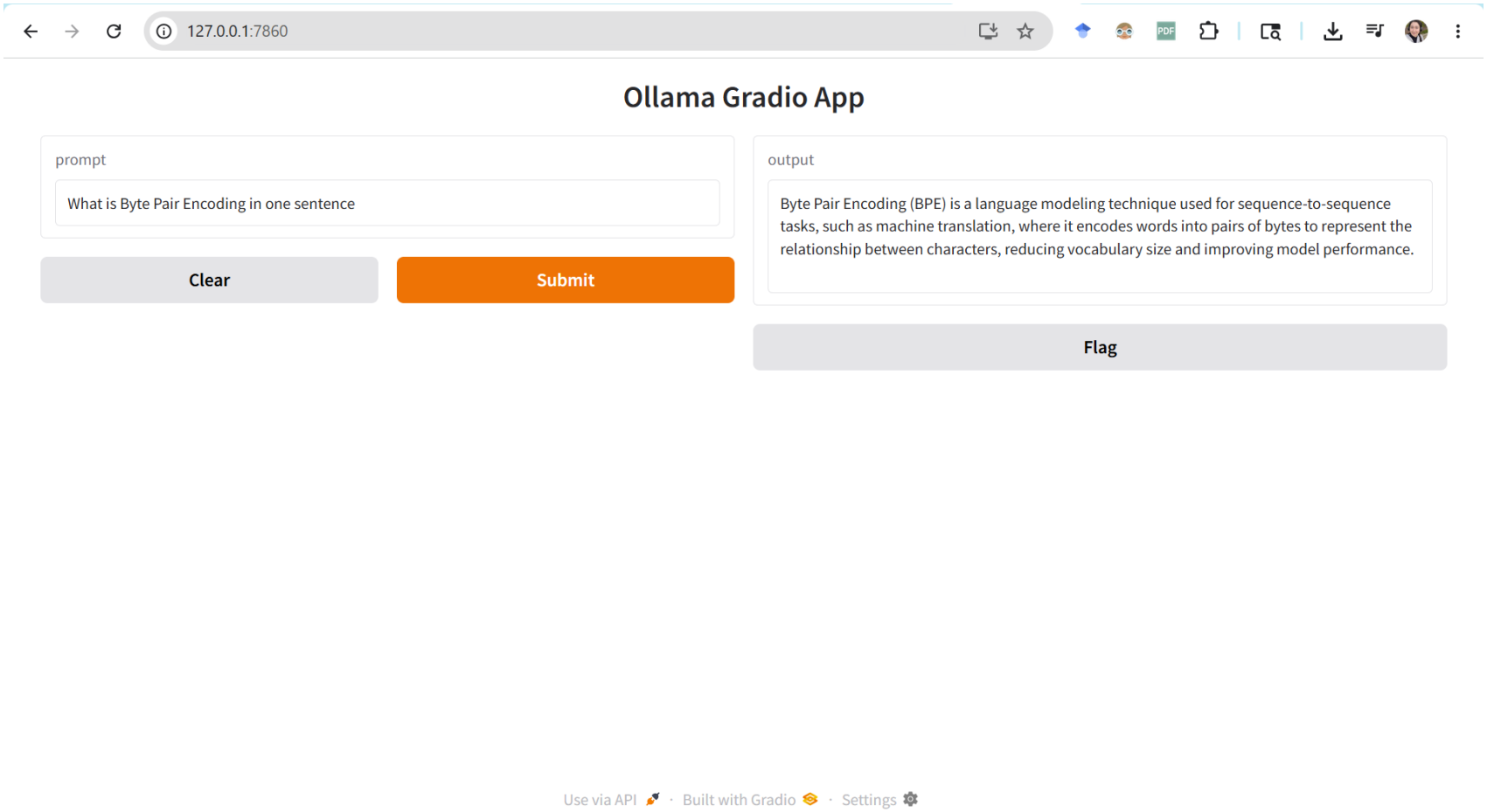
Gradio will open a local web link in the terminal.

```
PS C:\Users\Aurora> python C:\Users\Aurora\Desktop\NLP2\app1.py
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
```

Open the link in a web browser.

Then type a question and send it to the model.

Result



If you flag an output, it will be saved in the specified directory.

Current Subsection

1. Ollama
2. User Interface
   - Streamlit
   - Gradio
   - chainlit

Chainlit

Chainlit is a Python library for building chatbot-style AI apps.

It is useful when we want a real chat interface instead of a simple text box.

Good for:

- Chatbots
- LLM demos
- Prompt testing
- Multi-turn conversations

Installing Chainlit

First, install Chainlit in your terminal:

```
python -m pip install -user chainlit
```

Make sure Ollama is installed and running.

Also make sure the model is downloaded:

```
ollama pull llama3.2
```

Chainlit App with Ollama

Save this code as `app.py`.

```python
import chainlit as cl
import ollama

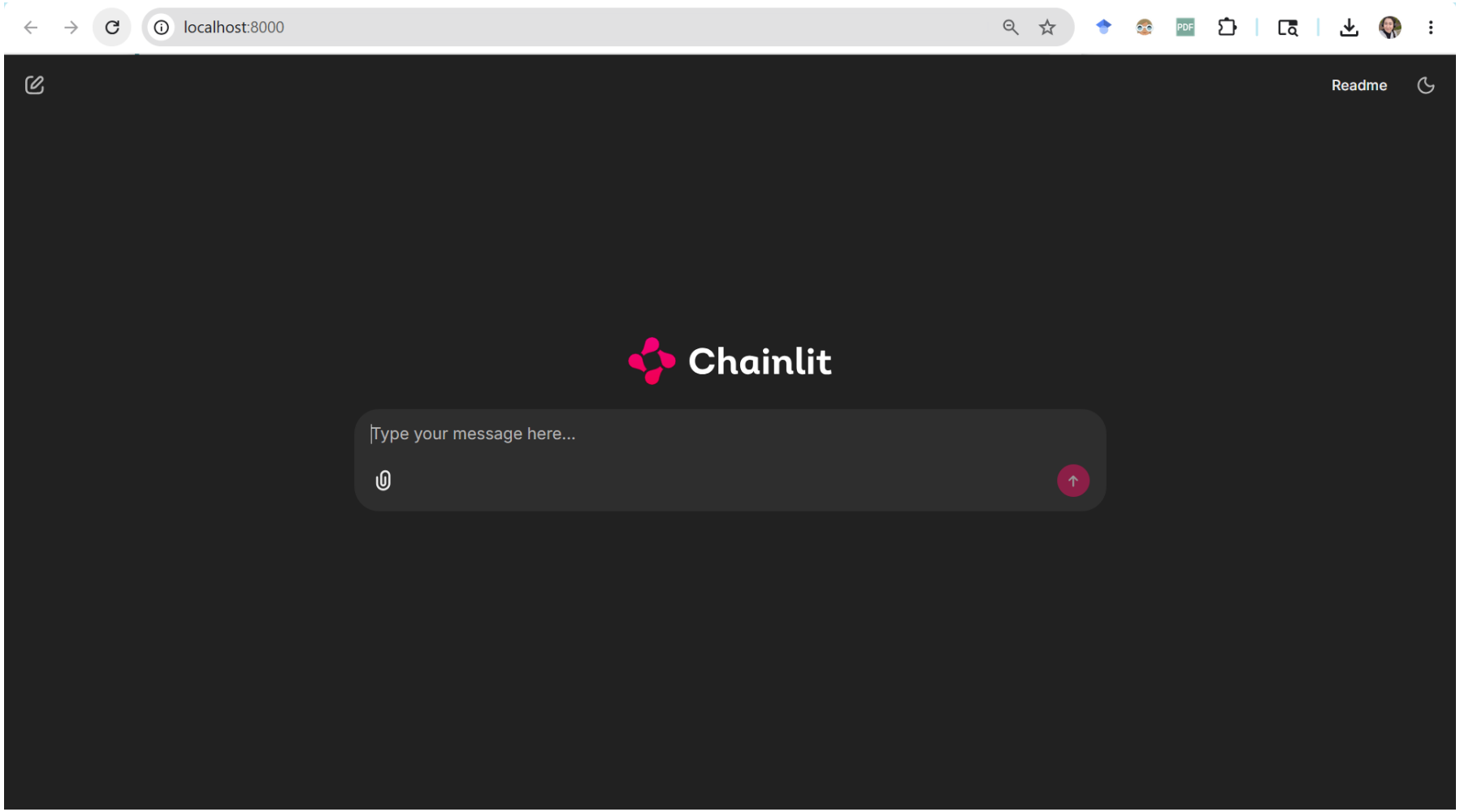
@cl.on_message
async def main(message: cl.Message):
    response = ollama.generate(
        model="llama3.2",
        prompt=message.content
    )

    await cl.Message(
        content=response["response"]
    ).send()
```

Run it:

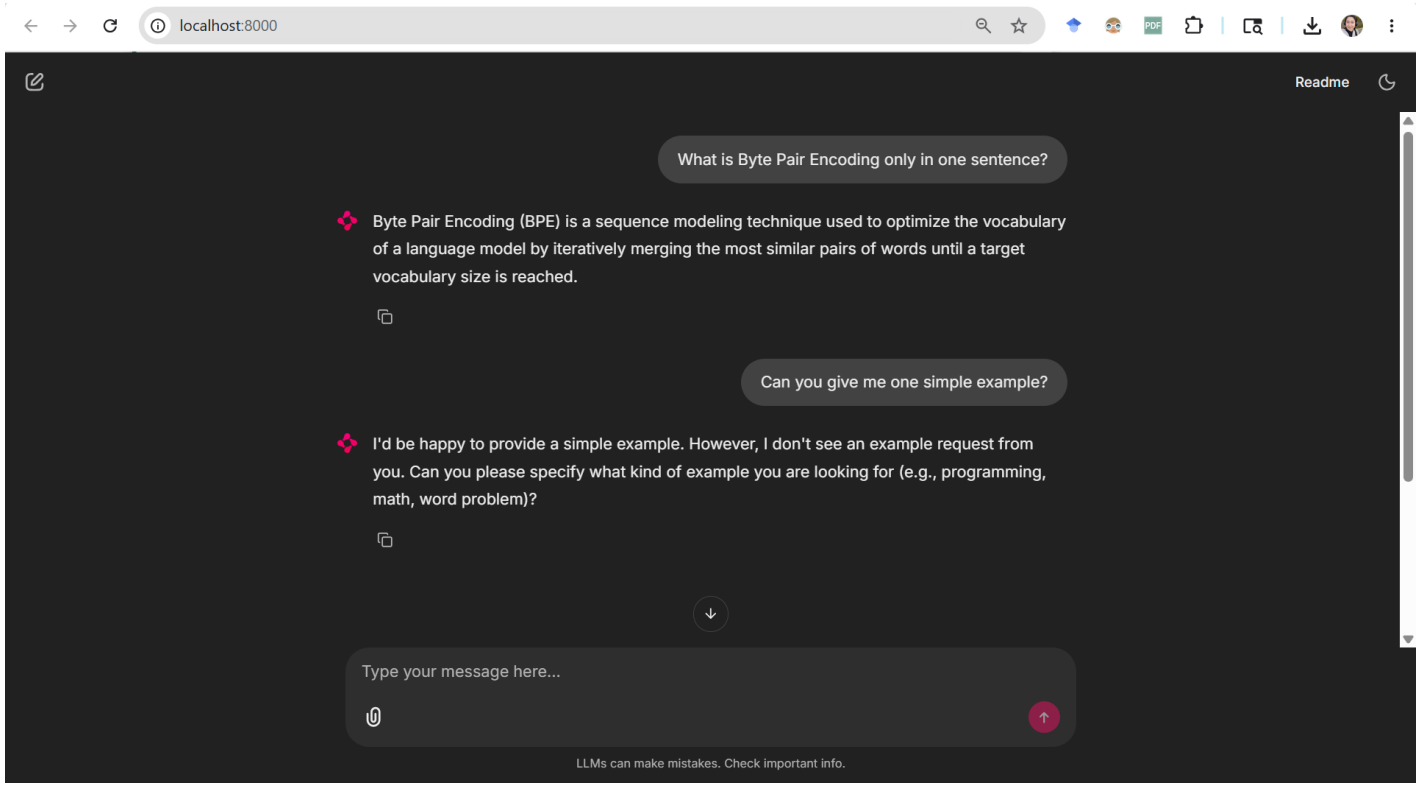
```
python -m chainlit run C:\Users\Aurora\Desktop\NLP2\app2.py
```

Result



Example

Right now, we are only getting a result for each individual message, so the model does not have access to the previous conversation history, as shown in this example:



Using the Assistant Role

Right now, each message is sent to the model separately.

This means the model does not remember the previous messages.

To fix this, we store the full conversation history (using assistant role).

Each new conversation turn is saved as:

- `user`: the message written by the user
- `assistant`: the response generated by the model

Then we send the full message history back to Ollama each time.

This gives the model access to the previous conversation.

Code

```python
import chainlit as cl
import ollama

@cl.on_chat_start
async def start():
    cl.user_session.set("messages", [
        {"role": "system", "content": "You are a helpful teaching assistant. Explain clearly and simply."}
    ])

@cl.on_message
async def main(message: cl.Message):
    messages = cl.user_session.get("messages")

    messages.append({"role": "user", "content": message.content})

    response = ollama.chat(model="llama3.2", messages=messages)

    assistant_reply = response["message"]["content"]

    messages.append({"role": "assistant", "content": assistant_reply})

    cl.user_session.set("messages", messages)

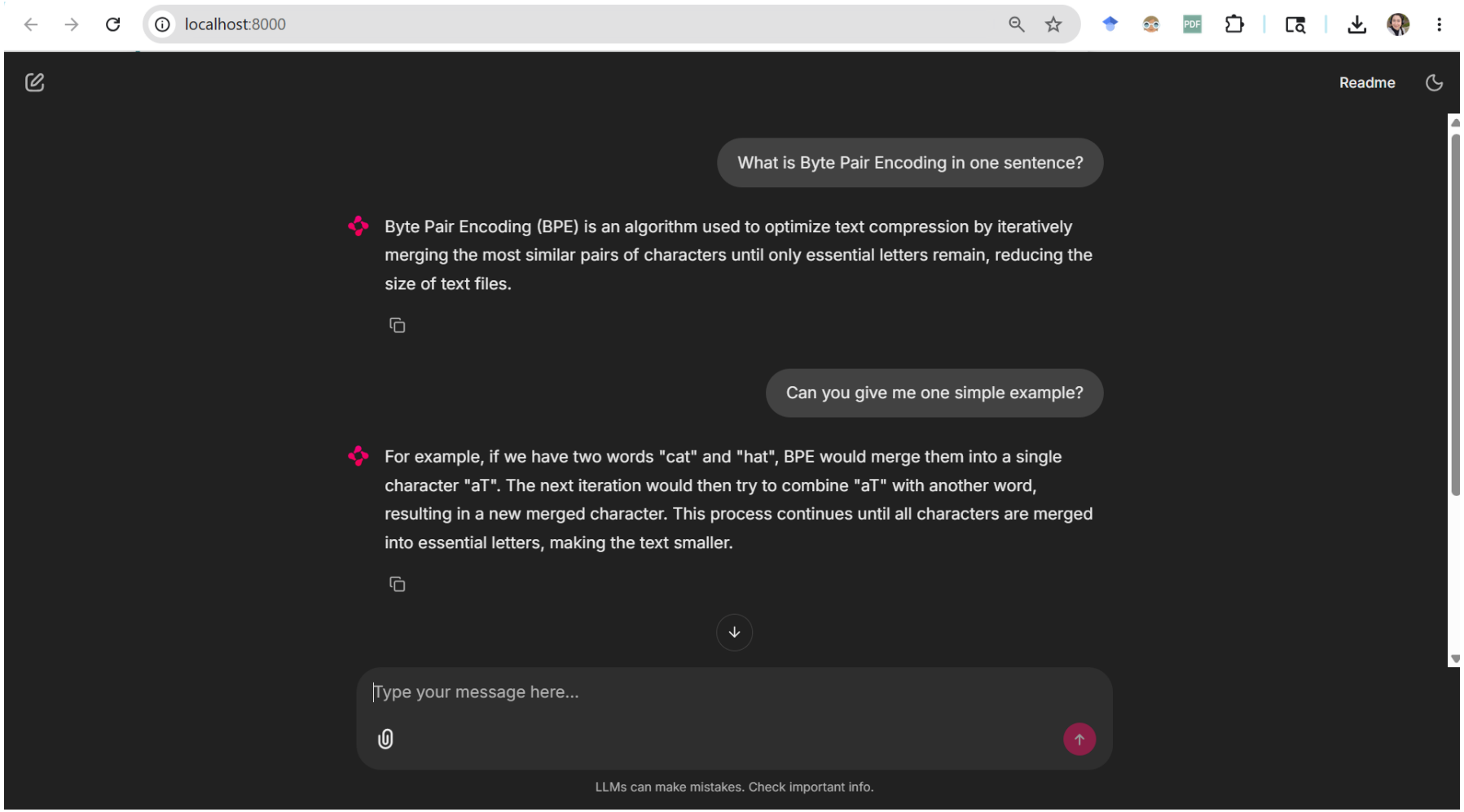
    await cl.Message(content=assistant_reply).send()
```

Save and run it as app3.py:

```
python -m chainlit run C:\Users\Aurora\Desktop\NLP2\app3.py
```

Problem Solved!

Interestingly, the model now has access to the conversation history:



Why Chainlit?

Chainlit gives us a chat interface with very little code.

The flow is simple:

- The user types a message.
- The Python function receives the message.
- Python sends the message to Ollama.
- Ollama sends the message to the model.
- The model response is displayed in the chat.

In a multi-turn chat, we store previous model responses as `assistant` messages and send them back with the next `user` message. This gives the model access to the conversation context.

Limitation of Conversation History

Our code can keep appending messages to the conversation history.

However, the model does not have unlimited memory.

If the total length of the messages becomes larger than the model's context window, the model cannot process the full history.

This means older messages may need to be removed, shortened, or summarized.

Key idea: The app can store unlimited history, but the model can only use the part that fits inside its context window.

How Real LLM Apps Handle Long History

Real LLM apps do not send unlimited history to the model.

They usually manage long conversations using strategies such as:

- **Recent history**: keep only the latest messages
- **Summarization**: summarize older messages into a shorter form
- **Retrieval**: search old messages and send only the relevant parts
- **External storage**: save full history in a database or file

The app stores the full conversation, but the model only receives the most useful context that fits inside its context window.

Congratulations!

Now you know how to add a user interface to an Ollama model!

